

2주 차 보고서

(2025년 1 월 4 일 ~ 2026년 1 월 10 일)

작성자

배주환

이번 주 회의 내용

이번 주 한 일

SpinLock, Sleep, Event, condition_variable, future, promise, packaged_task 등 다양한 Lock 구현 방식에 대해 공부하고 실습을 해보았다.

```
class SpinLock
{
public:
    void lock()
    {
        // CAS (Compare-And-Swap)

        bool expected = false;
        bool desired = true;

        while ( _locked.compare_exchange_strong(expected, desired) == false)
        {
            expected = false;

            //this_thread::sleep_for(std::chrono::milliseconds(100));
            this_thread::sleep_for(0ms);
            //this_thread::yield();
        }
    }

    void unlock()
    {
        _locked.store(false);
    }

private:
    atomic<bool> _locked = false;
};
```

```
void Producer()
{
    while (true)
    {
        {
            unique_lock<mutex> lock(m);
            q.push(100);
        }

        ::SetEvent(handle);

        this_thread::sleep_for(100ms);
    }
}

void Consumer()
{
    while (true)
    {
        ::WaitForSingleObject(handle, INFINITE);

        unique_lock<mutex> lock(m);
        if (q.empty() == false)
        {
            int32 data = q.front();
            q.pop();
            cout << data << endl;
        }
    }
}

int main()
{
    // 카널 오브젝트
    // Usage Count
    // Signal(피관볼) / Non-Signal(활관볼) << bool
    // Auto / Manual << bool
    handle = ::CreateEvent(NULL, /*보안속성*/, FALSE, /*bManualReset*/, FALSE, /*bInitialState*/, NULL);

    thread t1(Producer);
    thread t2(Consumer);
}
```

```

while (true)
{
    // 1) Lock을 잡고
    // 2) 공유 변수 값을 수정
    // 3) Lock을 풀고
    // 4) 조건 변수를 통해 다른 스레드에게 통지

    {
        unique_lock<mutex> lock(m);
        q.push(100);
    }

    cv.notify_one();    // wait중인 스레드가 있으면 딱 한개만 깨운다

    //this_thread::sleep_for(100ms);
}

Consumer()
while (true)
{
    unique_lock<mutex> lock(m);
    cv.wait(lock, []() { return q.empty() == false; });
    // 1) Lock을 잡고
    // 2) 조건 확인
    // - 만족0 => 빠져 나와서 이어서 코드 진행
    // - 만족X => Lock을 풀어주고 대기 상태

    // 그런데 notify_one을 했으면 항상 조건식을 만족하는거 아닌가?
    // Spurious Wakeup(가짜 기상?)
    // notify_one을 할때 lock을 잡고 있는 것이 아니기 때문

    // std::future
    {
        // 1) deferred -> lazy evaluation 지연해서 실행하세요
        // 2) async -> 별도의 스레드를 만들어서 실행하세요
        // 3) deferred | async -> 둘 중 알아서 골라주세요

        // 언젠가 미래에 결과물을 뱉어줄거야
        std::future<int64> future = std::async(std::launch::async, Calculate);

        // TODO

        int64 sum = future.get(); // 결과물이 이제서야 필요하다!
    }

    // std::promise
    {
        // 미래(std::future)에 결과물을 반환해줄거라 약속(std::promise)해줘
        std::promise<string> promise;
        std::future<string> future = promise.get_future();

        thread t(PromiseWorker, std::move(promise));

        string message = future.get();
        cout << message << endl;

        t.join();
    }

    // std::packaged_task
    {
        std::packaged_task<int64(void)> task(Calculate);
        std::future<int64> future = task.get_future();

        std::thread t(TaskWorker, std::move(task));

        int64 sum = future.get();
        cout << sum << endl;

        t.join();
    }
}

```

간단하게 서버 프레임워크를 만들어서 DummyClient, GameServer, ServerCore로 분류해놓았다.

다음 주 할 일

프레임워크를 더 수정해가면서 컴퓨터 구조와 관련된 이론인 캐시, 메모리 모델, cpu 파이프라인에 대해 공부할 예정이다.

특이사항